



Universidad de Costa de Rica

Escuela de Ciencias de la Computación e Informática

CI-0128 - G3

Proyecto Integrador de Ingeniería de Software y Bases de Datos

Profesores:

Sleyter Angulo

Luis Mata

Conceptualización

Grupo: Pingponeros S.A.

Integrantes:

José Manuel Mora Zuñiga C35280

Carlos Alberto Obando López C35655

Sebastián Gómez Castro C39031

I Semestre

2026

Proyecto de Cargas Administrativas

Descripción del problema que el software busca resolver

El sistema actual de la Vicerrectoría de Administración es manual (a papel), lo que genera una gestión desorganizada e ineficiente de documentos, horas trabajadas y funciones del personal. Se requiere un sistema digital confiable que centralice y automatice estos procesos.

¿Quién tiene el problema?

- Unidad de Vicerrectoría de Administración
- Funcionarios gestionados por la UCR
- Departamentos, unidades, áreas y secciones académicas universitarias

¿Qué dificultades genera actualmente este problema?

- No existe un control efectivo de horas trabajadas por unidades, departamentos, áreas y secciones dentro de la universidad.
- Los funcionarios no cuentan con una forma sencilla para actualizar sus horas laboradas ni sus funciones asignadas.
- La gestión manual de documentos genera desorden, inconsistencias y riesgo de pérdida de información crítica.

Contexto del problema

Contexto técnico

Todo se maneja a papel, después se digitalizan en una hoja de Excel.

Contexto social

Los funcionarios deben ir hasta la oficina de vicerrectoría para poder hacer los trámites en físico.

Contexto de negocio

Los errores derivados del proceso manual pueden afectar negativamente los ingresos y métricas de los funcionarios y las diferentes departamentos, unidades, áreas y secciones de la Universidad

Alcance del Proyecto

Stakeholder	Rol o interés en el sistema
Vicerrectoría de Administración	Product Owner
Funcionarios / Usuarios	Usuarios del sistema
Intermediarios (Profesores del P.I.)	Representan a Vicerrectoría ante el equipo de desarrollo

Definiciones para los requerimientos

“Administrador”: System Admin.

“Funcionario”: Usuario Final.

“Usuarios”: Todos los usuarios, ambos roles juntos.

Requerimientos funcionales

Gestión de usuarios

- a. El sistema debe permitir a los usuarios administradores poder registrar nuevos usuarios, proporcionando primer nombre, segundo nombre, primer apellido, segundo apellido, correo electrónico institucional y número de plaza, y asignarle su rol correspondiente.
 1. Se generará una contraseña temporal la cual el usuario deberá cambiarla antes de que se cumplan 48 horas, si esto pasa el usuario pasará al estado inactivo.
 2. Cuando se crea el usuario, le llegará al correo institucional la contraseña temporal.
- b. El sistema permitirá a los usuarios declarar la siguiente información general: Puesto de trabajo, clase ocupacional, cargo del puesto, jornada laboral y horario de trabajo.
- c. El sistema debe permitir a los usuarios iniciar sesión en su cuenta con las respectivas credenciales.
 1. Se mantendrá la sesión abierta durante 1 hora por medio de sessionStorage.
- d. El sistema permitirá a los usuarios tener la opción de recuperar la contraseña.
- e. El sistema permitirá a los usuarios cerrar sesión.
- f. El sistema permitirá a los usuarios administradores consultar la información no sensible (nombre, foto, clase, puesto y correo) de cualquier usuario en específico.
- g. Los usuarios administradores podrán acceder a perfiles de los usuarios funcionarios los cuales mostrarán información no sensible (nombre, foto, clase, puesto y correo).
- h. El sistema permitirá al usuario funcionario consultar toda la información relacionado con el mismo.
- i. El sistema permitirá al usuario asignar sus horas correspondientes a sus funciones y sus tiempos de descanso. Hasta 1.5 veces su jornada laboral (tiempo completo, medio tiempo...)
 1. El sistema trabajará internamente con base semanal y medirá los tiempos en minutos.

Gestión de cargas

- j. El sistema permite la asignación de un listado de funciones pertenecientes a un puesto de trabajo, de acuerdo con la información proveniente de la oficina de Recursos Humanos (lista será brindada).
- k. El sistema permite a los usuarios modificar la cantidad de horas de una función de un puesto de trabajo.
- l. El sistema permite a los usuarios agregar más funciones a su lista de funciones preestablecidas para ese puesto dentro de la unidad de trabajo.
- m. El sistema permite a los usuarios crear una nueva función de su puesto de trabajo, cuando esta función no se encuentre disponible dentro del listado de funciones (proveniente de la oficina de Recursos Humanos).
- n. El sistema permitirá a los administradores poder consultar la lista de funciones de un funcionario.
- o. El sistema permite al usuario ver el total de horas reportadas semanales, este es un cálculo que se realiza de forma automática.
- p. El sistema permite al usuario asignar la cantidad de horas dedicadas a tiempos de descanso, como: almuerzo, café (total), permisos o licencias autorizadas (cuál, cuánto tiempo semanalmente).
- q. El sistema permite al usuario asignar tiempo adicional (fuera de su jornada laboral), para desempeñar sus funciones, hasta un máximo de 1.5 veces su jornada laboral.
- r. El sistema permite al usuario justificar detalladamente cuánto tiempo y por qué utiliza ese tiempo (fuera de la jornada laboral) para esas funciones.
- s. El sistema permite al usuario identificar (indicar/declarar) si la jefatura inmediata conoce que el usuario está utilizando ese tiempo adicional.

Gestión de área, departamento, sección, y unidad

- t. El sistema permite a los administradores poder gestionar las áreas:
 - 1. Crear un área.
 - 2. Eliminar un área.
 - 3. Ver un área.
 - 4. Actualizar un área.
- u. El sistema permite a los administradores poder gestionar las secciones:
 - 1. Crear una sección.
 - 2. Actualizar una sección.
 - 3. Ver una sección.
 - 4. Eliminar una sección.
- v. El sistema permite a los administradores poder gestionar los departamentos:

1. Crear un departamento.
 2. Actualizar un departamento.
 3. Ver un departamento.
 4. Eliminar un departamento.
- w. El sistema permite a los administradores poder gestionar las unidades:
1. Crear una unidad.
 2. Actualizar una unidad.
 3. Ver una unidad.
 4. Eliminar una unidad.

Gestión de puesto

- x. El sistema permite a los administradores poder gestionar los puestos:
1. Crear un puesto.
 2. Actualizar un puesto.
 3. Ver un puesto.
 4. Eliminar un puesto.
- y. El sistema permite a los administradores poder gestionar la información de los puestos como: nombre del puesto, descripción del puesto, y tipo de puesto.
- z. El sistema permite a los administradores poder gestionar los tipos de puestos (propios de mi puesto, de otro puesto, y de apoyo ocasional):
1. Crear un tipo de puesto.
 2. Actualizar un tipo de puesto.
 3. Ver un tipo de puesto.
 4. Eliminar un tipo de puesto.

Gestión de función

- aa. El sistema permite a los administradores poder gestionar las funciones y su información: nombre, descripción, tipo de puesto, periodicidad de la función (diaria, semanal, quincenal, mensual, semestral, y anual)
1. Crear una función.
 2. Actualizar una función.
 3. Ver una función.
 4. Eliminar una función.

Reportes

- bb. El sistema permite a los administradores generar distintos tipos de informes en formato Excel (.xlsx) y PDF (.pdf).

1. Informe de información general de un funcionario: Nombre completo, número de plaza, clase ocupacional, cargo de puesto, jornada y horario laboral
 2. Informe de horarios de trabajo por área, departamento, sección, o unidad.
 3. Informe de horarios de trabajos fuera de jornada laboral por área, departamento, sección, o unidad.
- cc. El sistema permite a los usuarios generar un reporte de la información que se acaba de cargar de las horas registradas en las funciones de su puesto, en formato PDF.

Requerimientos no funcionales

Usabilidad

- a. Los estudiantes deben desarrollar un sistema que se adapte a los distintos tamaños de pantallas de los dispositivos (móviles y escritorio) mediante diseño responsivo.
- b. Los estudiantes deben diseñar las interfaces del sistema utilizando previamente prototipos de alta fidelidad en herramientas como Figma, asegurando coherencia visual y funcional con el producto final.
- c. Los estudiantes deben aplicar principios actuales de diseño de interfaces gráficas de usuario (UI) y experiencia de usuario (UX), basándose en tendencias y buenas prácticas documentadas.
- d. El sistema debe garantizar que cualquier usuario pueda completar las tareas principales en un máximo de tres clics desde la pantalla principal.
- e. Los estudiantes deben asegurar consistencia en colores, tipografías, iconografía y estilos en todas las pantallas del sistema.
- f. El sistema debe proporcionar mensajes de error claros, comprensibles y orientados a la solución cuando ocurra una falla.
- g. El sistema debe incluir validaciones en tiempo real en los formularios para mejorar la experiencia del usuario.
- h. La interfaz debe cumplir con criterios básicos de accesibilidad, como contraste adecuado de colores y uso de etiquetas descriptivas en los campos de entrada.

Seguridad

- i. Los estudiantes deben asegurar el uso de buenas prácticas de programación para evitar problemas como inyección SQL, inyección de código y ataques de tipo XSS (Cross-Site Scripting).
- j. El sistema debe implementar autenticación segura mediante el uso de contraseñas cifradas utilizando algoritmos de hash robustos.
- k. El sistema debe implementar autorización basada en roles para restringir el acceso a funcionalidades según el tipo de usuario.

- l. El sistema debe gestionar adecuadamente las sesiones de usuario, incluyendo expiración automática tras un periodo de 1 hora de inactividad.
- m. El sistema debe validar tanto en el cliente como en el servidor todos los datos ingresados por el usuario.
- n. El código fuente debe almacenarse en un sistema de control de versiones con control de acceso restringido.

Rendimiento

- o. El sistema debe ser capaz de manejar al menos 100 usuarios concurrentes sin degradación significativa del rendimiento.
- p. Los estudiantes deben optimizar las consultas a la base de datos mediante el uso adecuado de índices.
- q. El sistema debe minimizar el tamaño de los recursos estáticos (imágenes, archivos CSS y JavaScript) mediante técnicas de compresión y minificación.
- r. El sistema debe implementar paginación en listados extensos para reducir la carga en el servidor.
- s. Los estudiantes deben realizar pruebas de rendimiento antes de la entrega final del proyecto.

Escalabilidad

- t. El sistema debe diseñarse siguiendo una arquitectura modular que permita la incorporación de nuevas funcionalidades sin afectar significativamente el código existente.
- u. Los estudiantes deben aplicar principios de diseño como separación de responsabilidades (por ejemplo, arquitectura MVC o similar).
- v. La base de datos debe diseñarse con normalización adecuada para facilitar su crecimiento futuro.
- w. El sistema debe permitir la migración a entornos de mayor capacidad (por ejemplo, servidores en la nube) sin requerir una reestructuración completa del código.
- x. Los estudiantes deben documentar la arquitectura del sistema para facilitar futuras ampliaciones o mantenimiento.
- y. El sistema debe permitir escalar horizontalmente mediante la posibilidad de desplegar múltiples instancias del servidor si fuera necesario.

3. Restricciones del proyecto

Restricciones (delimitador)

Incluye:

- Registro de usuarios
- Manejo de cargas administrativas
- Reportes con base en la información anterior (información general de un usuario, informe por horas trabajadas entre áreas, informe de horas trabajadas fuera del horario laboral)

No incluye:

- Firma digital
- Aplicación móvil

4. Riesgos del proyecto

Riesgo iniciales	Posible impacto
Información incompleta	Imposibilidad de conocer el alcance real del proyecto.
Falta de tiempo	No se puede plantear el diseño estructural del proyecto.
Falta de personal	Tener solo 3 miembros limita el desempeño del equipo.

Riesgos en el desarrollo del producto	Posible impacto

Riesgos finales	Posible impacto

5. Plan de Calidad

Estándares de código

- Nombres descriptivos
- Comentarios (documentación exhaustiva)

- Seguir el linter de cada lenguaje (y las nomenclaturas para nombres de variables, funciones, clases, etc.)

Tipos de pruebas que se realizarán

- Pruebas unitarias
- Pruebas de integración
- Pruebas manuales
- Pruebas con usuarios

Criterios para la revisión de pares

Que pase los test, el linter y otros chequeos necesarios.

Repositorio utilizado y ramas

Herramienta: **GitHub**

Ramas:

main — es la rama que va a tener el producto funcional.

- Contiene únicamente código listo para producción.
- Nunca se hace commit directo aquí.
- Cada merge representa una versión de sprint que se desarrolló del producto.
- Etiquetar cada merge con un tag de versión: v1.0.0, v2.0., etc.

dev — la rama en desarrollo para los sprints.

- Es la rama de integración continua.
- Aquí se fusionan todas las features terminadas antes de pasar a main.
- Debe mantenerse siempre en un estado estable y funcional.
- Al cierre de un sprint, dev se mergea a main si todo está aprobado con sus respectivas pruebas y validación de la estructura y escritura del código fuente .

Para desarrollar nuevas funcionalidades (feature): feature/nombre-descriptivo, ejemplo: dev → feature/login → dev .

Para corregir errores detectados en el desarrollo (en dev, no en main): bugfix/nombre-descriptivo, ejemplo: dev → bugfix/login_crash → dev .

Hotfix?

Reglas de Oro

- Nunca hacer push directo a main o dev.
- Todo cambio entra por Pull Request con al menos una revisión y aprobado por parte de un miembro del equipo.
- Las ramas de trabajo deben ser de vida corta (idealmente dentro del sprint).
- Antes de abrir un PR, la rama debe estar actualizada con dev para evitar conflictos.
- Eliminar la rama después del merge para mantener el repositorio limpio.

Herramientas de análisis estático y justificación

- **ESLint + Prettier** (Frontend)
 - Estándar para **React**.
- **Roslyn Analyzers** (Backend)
 - Built-in para **C#**.
- SonarQube / **SonarCloud** (Ambos)
 - Para chequeos unificados.